SOFTWARE TESTING

STRATEGIES

Presented by
Rifa Zumana
Lecturer
Dept. of CSE, NBIU

STRATEGIC APPROACH TO SOFTWARE TESTING

Module-First

Testing begins at the module level and works outward towards integration of the entire computer-based system.

Technique-Adaptive

Different testing techniques are applied at different points in time throughout the process.

Dual Responsibility

Testing is conducted by the software developer and an Independent Test Group (ITG) for large projects.

Testing ≠ Debugging

Debugging is distinct from testing but is essential in any effective testing strategy.

VERIFICATION & VALIDATION

VERIFICATION

Does the product meet its specifications?
"Are we building the product right?"

VALIDATION

Does the product perform as desired?
"Are we building the right product?"

Verification and Validation

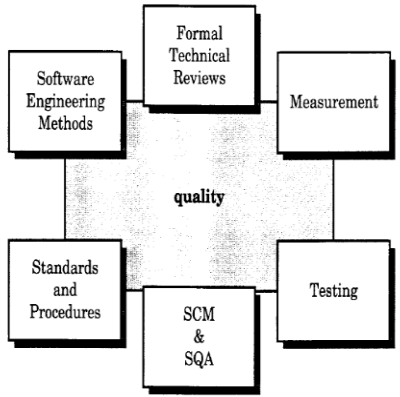


FIGURE 17.1. Achieving software quality

Verification -- Does the product meet its specifications? Are we building the product right?

Validation -- Does the product perform as desired? Are we building the right product?

SOFTWARE TESTING STRATEGY

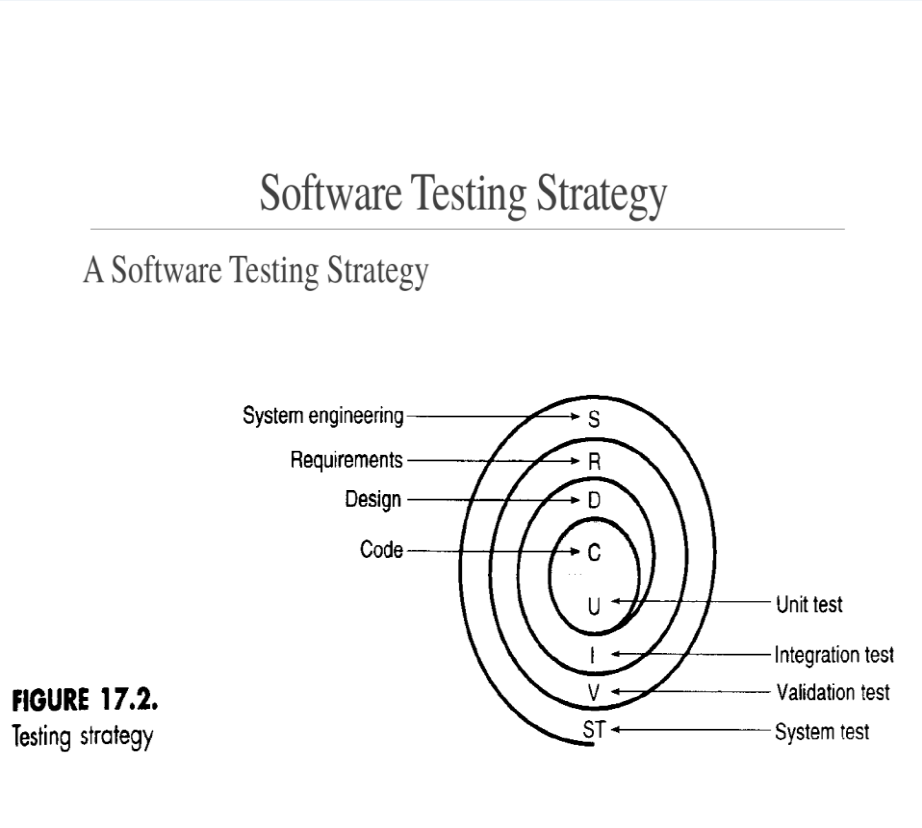


FIGURE 17.2.
Testing strategy

Unit Test

Tests individual modules/functions in isolation.

Integration Test

Tests interaction between combined modules.

Validation Test

Confirms software meets all requirements.

System Test

Tests the entire integrated system end-to-end.

UNIT TESTING

Focuses on the smallest element of software design — the module.

Interface

- No. of input parameters matches arguments
- Parameter attributes and units match
- Global variable definitions consistent

Data Structures

- Improper or inconsistent typing
- Erroneous initialization or defaults
- Underflow, overflow, addressing exceptions

Boundary Conditions

- All execution paths covered
- Arithmetic & initialization errors
- Precision inaccuracies checked

Error Handling

- Exception handling correctness
- Error messages are intelligible
- System interrupt behavior tested

INTEGRATION TESTING

Systematic construction of the program while uncovering interface errors

TOP-DOWN INTEGRATION

✓ Advantages

- Verifies major control/decision points early
- Complete function demonstrable with depth-first
- Confidence builder for developer & customer

✗ Disadvantages

- No significant data can flow upwards via stubs

BOTTOM-UP INTEGRATION

✓ Advantages

- Easier test case design
- No stubs needed

✗ Disadvantages

- Program as entity doesn't exist until last module is added

Sandwich Testing: Combined top-down for upper levels + bottom-up for lower levels

INTEGRATION TESTING — VISUAL APPROACHES

Top-Down Integration

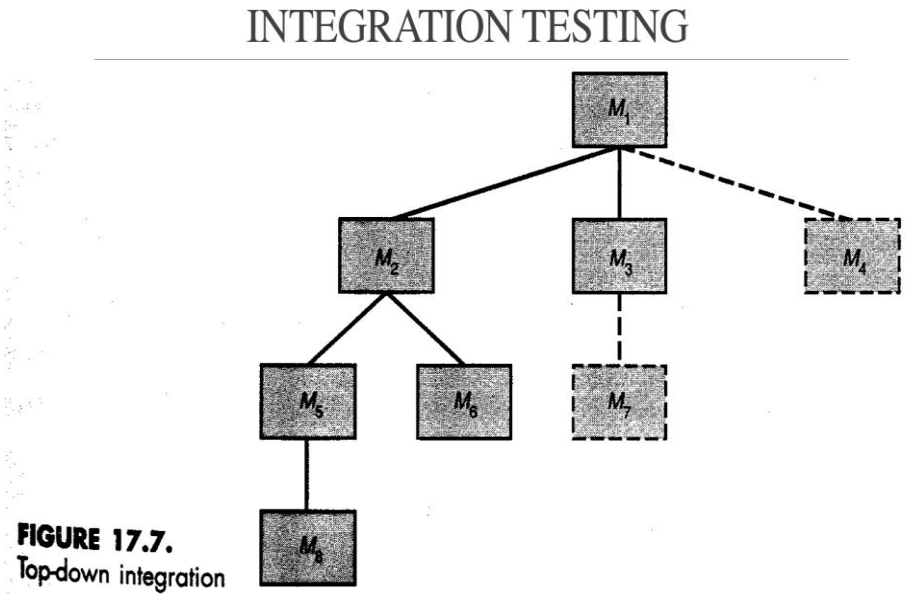


FIGURE 17.7. Top-down integration

Bottom-Up Integration

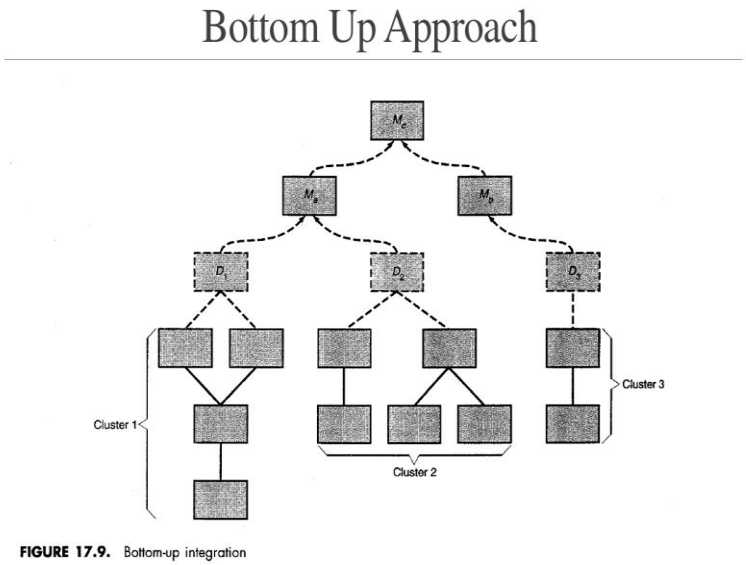


FIGURE 17.9. Bottom-up integration

REGRESSION & VALIDATION TESTING

REGRESSION TESTING

Re-execution of tests to ensure new changes have no unintended side effects.

Test Suite Must Include:

- Representative tests exercising ALL software functions
- Tests focusing on functions likely to be affected by the change
- Tests focused on software components that have changed

VALIDATION TESTING

Final assurance that software meets ALL functional, behavioral, and performance requirements.

Uses exclusively black-box testing techniques

Alpha & Beta Testing:

Alpha Test: Conducted at developer's site, by the customer

Beta Test: At customer's site in a live environment

SYSTEM TESTING

Verifies that all system elements have been properly integrated and perform as required.

Recovery Testing

Forces software to fail in various ways and verifies that recovery is properly performed. Assesses how the system responds to crashes, data loss, or hardware failures.

Security Testing

Attempts to verify the protection mechanisms. The goal: making penetration cost more than the value of information obtained by breaking in.

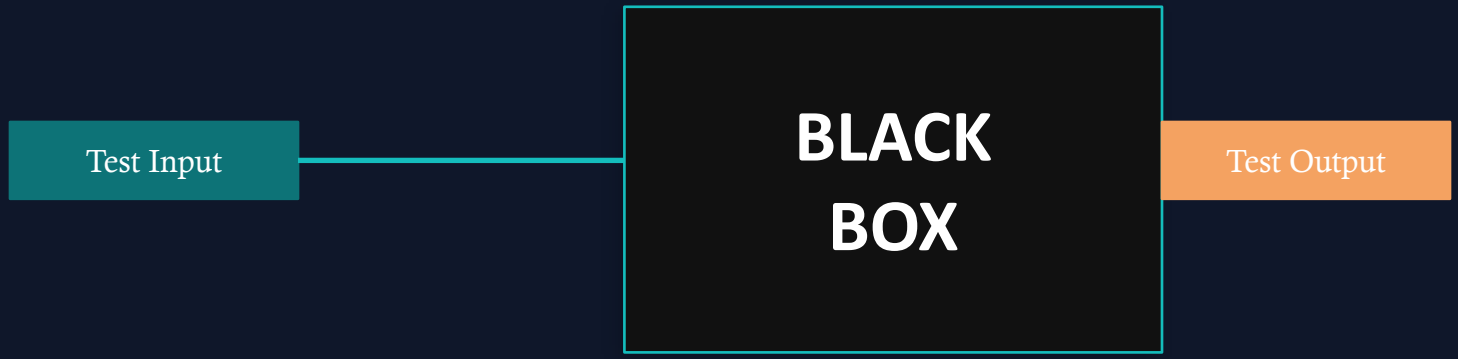
Stress Testing

Executes the system in a manner that demands resources in abnormal quantity, frequency, or volume — pushing the system beyond normal operational capacity.

Performance Testing

Tests run-time performance of a system within the context of an integrated system. Evaluates response time, throughput, and resource usage.

BLACK BOX TESTING



Internal logic is INVISIBLE to the tester

- Tests software from the user's perspective — no knowledge of internal code
- Focuses on inputs and expected outputs
- Used extensively in Validation Testing
- Techniques: Equivalence Partitioning, Boundary Value Analysis
- Can be performed by testers without programming knowledge

WHITE BOX TESTING

```
if (x > 0) {
  return A;
}
else {
  return B;
}
```

Internal logic is VISIBLE to the tester

- Also called Glass Box or Structural Testing
- Tester has full knowledge of internal code and structure
- Used heavily in Unit Testing
- Tests all paths, branches, and conditions in source code
- Techniques: Statement Coverage, Branch Coverage, Path Testing

GRAY BOX TESTING

Partial knowledge of internal structure

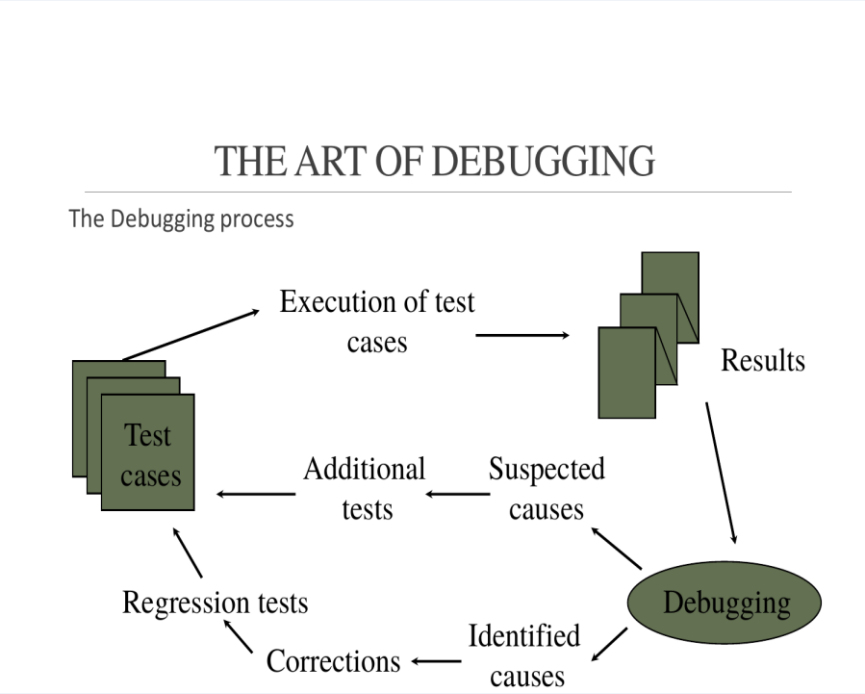
- Combines aspects of both Black Box and White Box testing
- Tester has limited/partial knowledge of internal workings
- Effective for integration testing and web application testing
- Tests both functionality AND internal implementation concerns
- Helps identify context-specific issues that pure black/white box might miss

BLACK vs WHITE vs GRAY BOX COMPARISON

Criteria	Black Box	White Box	Gray Box
Knowledge	None	Full	Partial
Perspective	User / External	Developer / Internal	Integration / Both
Testing Focus	Functionality	Code Structure & Paths	Both Aspects
Used In	Validation, System	Unit Testing	Integration Testing
Technique	Equiv. Partitioning, BVA	Coverage, Path Testing	Hybrid Techniques
Who Tests	Any Tester	Developer Required	Developer or Tester

THE ART OF DEBUGGING

Debugging is a consequence of successful testing. When a test uncovers an error, the debugging process locates and removes it.



Brute Force

Memory dumps, runtime traces. Least efficient but most common approach.

Backtracking

Once error is found, trace backward to locate the root cause.

Cause Elimination

Isolate potential causes, develop hypotheses, and test to isolate the bug.

Debugging Tools

Use automated tools, debuggers, and profilers to assist diagnosis.

Thank You